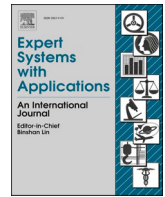




Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa



SP-ant: An ant colony optimization based operator scheduler for high performance distributed stream processing on heterogeneous clusters

Mohammadreza Farrokh, Hamid Hadian, Mohsen Sharifi^{*}, Ali Jafari

Distributed Systems Research, Lab University of Science and Technology, Tehran, Iran

ARTICLE INFO

Keywords:

Ant colony optimization
Distributed data stream processing
Scheduling
Apache Storm
Heterogeneous clusters

ABSTRACT

A key feature of distributed stream processing (DSP) systems is the scheduling of operators on clustered computers. In scheduling, the assignment plan of operators to nodes of the cluster, requirements of operators, and the computational power of each worker node must be considered with the goal of finding a tradeoff between the communication latency of operators and the utilization of worker nodes to minimize the overall system response time. To reach this goal is quite challenging, especially in heterogeneous clusters, because there are no accurate estimations about the loads of worker nodes at run time. To address this challenge, we propose a novel stream processing scheduling using ant colony algorithm (SP-Ant). SP-Ant finds the best operator assignment plan considering the inter-node communication latencies of operators by initially collocating highly communicative operators on the same worker nodes using the bin-packing algorithm and iteratively (re-)scheduling only the less communicative operators using the exploration and exploitation phases of the evolutionary ant colony optimization (ACO) algorithm in order to reduce its convergence time. SP-Ant is implemented on the standard Apache Storm. Using several standard benchmark topologies of Storm, it is shown that SP-Ant outperforms the R-Storm and Storm default schedulers by at least 50% in reducing the response time.

1. Introduction

As time passes, an exponential increase of data generated by IoT infrastructures such as wearable devices, traffic monitoring as well as real-time detection of complex patterns are witnessed (Fardbastani and Sharifi, 2019). The need to process these data in a real-time manner is crucial, and until now, many solutions have been proposed. The early generations of data stream processing were some sort of database management systems that could handle heavy workloads in a distributed manner. There had been SQL-like commands to write queries to handle data. Batch processing was another solution to overcome Hadoop's problem [1]. Stream processing with minimum latency has motivated the development of new generations of distributed stream processing (DSP) systems. Under heavy workloads, scalability, fault tolerance, and short response time become critical to such systems.

In DSP systems, the users' computing elements (operators) can be considered as vertices of a graph connected through the edges, which are streams. This graph includes three types of computing elements; source, operators, and sink, which account for data generators, processing

elements, and data consumers, respectively. The other aspect of data stream processing platforms is an application programming interface (API), enabling users to only work with a series of functions without knowing the details of engine design. Some of the popular DSP systems are Apache Storm, Apache Flink, and Apache Spark.

Apache Storm is one of the most well-known stream processing systems used to process data in distributed environments with high performance. It is reported that the problem of scheduling and operator placement in DSP systems is an NP-hard problem, and that for large-scale clusters, efficient heuristic algorithms are required (Nardelli et al., 2019; Röger and Ruben, 2019).

Some schedulers in DSP systems firstly analyze and obtain an initial estimation about the required resources of operators in topology and the capacity of worker nodes in the cluster before running the system and deploying the operators. In contrast, some other schedulers permanently monitor the system's performance at runtime and, based on the result, relocate the operators or increase the degree of parallelization (Aniello et al., 2013). A salient feature of data stream processing is scalability or the ability to increase or decrease operators' instances in case of heavy

^{*} Corresponding author.

E-mail addresses: Mo_farrokh@comp.iust.ac.ir (M. Farrokh), hadian_hamid@cmps2.iust.ac.ir (H. Hadian), msharifi@iust.ac.ir (M. Sharifi), ali_jafari@comp.iust.ac.ir (A. Jafari).

<https://doi.org/10.1016/j.eswa.2021.116322>

Received 7 January 2021; Received in revised form 16 November 2021; Accepted 26 November 2021

Available online 3 December 2021

0957-4174/© 2021 Elsevier Ltd. All rights reserved.

workloads. For operator placement and scheduling of operators, various algorithms such as greedy strategies (Peng et al., 2015), integer linear programming (Cardellini et al., 2017), heuristics (Liu & Buyya, 2017), and meta-heuristic methods (Smirnov et al., 2017) have been used.

Elasticity and scaling are the adaptation at runtime and should be done with low latency. Nevertheless, performing changes in operators' assignments while the system is running would bring about negative consequences such as tuple process failure and system downtime, which seriously degrade the quality of service (QoS) metrics. On the other hand, all the decisions for the system cannot be taken before starting it. Since many parameters, such as load variation, heterogeneity of resources, and operators' actual behavior, are all unknown before running, control theory (De Matteis & Mencagli, 2017), reinforcement learning (Russo et al., 2019), and queuing theory (Lohrmann et al., 2015) techniques have been used to guide decisions at runtime.

In this paper, we present the SP-Ant algorithm to address DSP systems' scheduling, which uses the ant colony optimization (ACO) algorithm, a well-known meta-heuristic, and evolutionary algorithm, to obtain high performance. The evolutionary algorithm aims to refine the decisions of an agent through subsequent steps. In the long-run, actions with more positive consequences would be selected via permanent changes in the reward values. Leveraging the ACO algorithm enables us to have runtime adaptation while scheduling operators. So far as we are aware, it is for the first time that this paper uses a combination of heuristic and meta-heuristic algorithms for operator scheduling in distributed data stream processing systems running on heterogeneous cluster of computers, resulting in considerable improvement in the topology response time.

In SP-Ant, a number of operators contribute to the scheduling, and the algorithm continues to evolve by getting real feedback from the already assigned operators on the worker nodes. The bin-packing algorithm (Peng et al., 2015; Liu & Buyya, 2017) is used for initial scheduling to reduce the operator's inter-node communications and the ACO algorithm's convergence time. A percentage of bins capacity is defined as a threshold for each bin, and the scheduler assigns operators to each worker node just equal to this threshold. SP-Ant excludes them from participating in the scheduling by collocating operators that are highly communicative on the same worker nodes. These operators have the highest communication latency in the bins, and by inhibiting them from further scheduling, we aim to decrease communication overheads. On the other hand, unassigned operators can contribute to the evolutionary algorithm to determine the exact number of operators in each bin and find each operator's optimal placement in the cluster. Hence, by following this policy, the number of operators in the ACO scheduling will decrease remarkably, which in turn can result in faster convergence time.

We can summarize the contributions of this paper as follows:

To address the scheduling of operators in DSP systems, SP-Ant exploits the ACO algorithm as an evolutionary algorithm, enabling to attain better runtime adaptation. Since our worker nodes are heterogeneous, and operators can be computationally, or I/O intensive, operators' optimal placement in topology would be found through various iterations by acquiring accurate estimation at runtime. Furthermore, the exact number of operators in worker nodes will be determined.

SP-Ant is a joint of the bin-packing algorithm and the ACO algorithm. SP-Ant considers communication between operators, and as a result, its performance is higher compared to when all operators are involved in the scheduling blindly. The bin-packing algorithm is used to decrease the communication overheads. In each bin, a number of operators are considered as pinned operators, and the remaining are fed to the ACO algorithm. In addition, the convergence time is significantly lower because a subset of the operators no longer participates in the ACO scheduling.

The rest of the paper is organized as follows. Section 2 provides a background review of stream processing, Apache Storm, and related works. Section 3 presents the design and implementation of the SP-Ant

scheduler. Section 4 reports our experiments' results, followed by a discussion on Section 5, and Section 6 concludes the paper alongside our future mandates.

2. Background and related work

Apache Storm is a state-of-the-art stream processing system by which a tremendous amount of data is processed with milliseconds latency in a distributed manner. Apache storm can provide scalability, fault tolerance, and availability for users. Storm provides APIs for users to easily develop stream processing applications, and many facilities such as a performance monitoring panel and an administrative tool are provided to change the resource management and the degree of parallelization.

Apache Storm can process one million tuples per second. This significant throughput is possible by using queuing and database systems technologies. Programmers write their programs and only submit them, and Storm converts each program to a topology in the form of a directed acyclic graph (DAG). At runtime, processing elements of this graph are spread over the worker nodes to attain good system performance.

The basic elements of Apache Storm include:

- Topology: The structure at which developers provide their programs and submit them to the Apache Storm.
- Tuple: A sequence of input data that has a timestamp and can have several fields.
- Executor: The smallest entity that can be scheduled in Storm and could run one or more tasks of the same operator.
- Tasks: The instances of each operator, including Bolts or Spouts.
- Worker process (Slot): A Java process that can run one or more executors (Cardellini et al., 2018).

In the Storm topology, there are two types of operator components, namely, *Spout* and *Bolt*. Spouts are generators of tuples. Furthermore, they can retrieve the information from an external source in different formats and forward it to the bolts. Bolts are actual processors of information. They can filter, aggregate, or modify the information and finally transfer it to the other bolts or sink. The sink is a bolt that eventually consumes processed data of topology.

Apache Storm has three basic architectural components, namely, *Nimbus*, *Supervisors*, and *Zookeeper*. Nimbus is the master node responsible for assigning operators to the worker nodes in the cluster and monitoring the system's performance and failures. Nimbus is a daemon process that runs when the system starts, and it uses Apache thrift service as a means of communication with supervisors. Nimbus is heavily reliant upon the Zookeeper for transferring messages from running operators to supervisors. Zookeeper serves as a shared memory that permanently stores the status of operators.

The supervisor is also a daemon process that runs on worker nodes and waits for Nimbus's commands to run a worker process. All coordination and communications between Nimbus and Supervisors are done through the Zookeeper. Each worker is a Java process and is responsible for running one or more executors. The scheduler module is located in Nimbus, and by submitting a topology, it assigns executors to different worker processes based on a pre-determined policy. The default scheduler of Storm uses a round-robin algorithm.

Concerning the scheduling, the DSP systems' main characteristic is that operators are statically planned to be assigned to nodes in the cluster before the system starts, and no operator leaves the system at runtime. The scheduling can be defined as an optimization problem trying to find a threshold for the number of operators that can be placed on a worker node to reduce inter-node communications. Moreover, by adding more operators, the worker node may be overutilized, degrading the system performance.

As stated before, DSP systems adopt two approaches for scheduling. In the first approach, the capacity of worker nodes and resource requirements of each operator is estimated before running the system,

while in the second approach, the main focus is on changing the parallelization degree of operators at runtime with variations in the workloads of worker nodes (Aniello et al., 2013).

Meta-heuristic algorithms are algorithms that are inspired by nature, physics and man and are used to solve many optimization problems. Meta-heuristic algorithms aim to provide a balance between the local optimum and the overall performance of the system. Meta-heuristic algorithms acquire the best result by exploring the search space and avoid local optimums by random jumps through the search space (Stanoi et al., 2007). Usually meta-heuristic algorithms are used in combination with other algorithms to achieve the optimal solution or to exit the local optimal state. Population based meta-heuristics have been the dominant methods to find optimal or near-optimal solutions to many optimization problems within a reasonable time (Karla et al. 2015). There are several meta-heuristic algorithms such as genetic algorithms (GAs), particle swarm (PSO) (Osaba et al., 2019), ant colony (ACO), grey wolf (GWO) (Precup et al., 2016) and bat algorithm (Zapata et al., 2020). These algorithms among others have been used to solve real world problems in several domains.

(Bilal & Canini, 2017) have tried to optimize streaming application performance by leveraging the Hill-climbing algorithm, a metaheuristic algorithm. They have automated the initial configuration (e.g., operator replication) of the system by parameter tuning. This type of initial configuration is however prone to system performance degradation at runtime.

(Nardelli et al., 2019) have considered the problem of scheduling and operator placement as a multi-objective optimization problem. They have used various meta-heuristic algorithms such as local search and tabu search to increase operator placement efficiency. They have considered a combination of availability, application response time, and network usage as their QoS.

(Smirnov et al., 2017) have used the genetic algorithm as an evolutionary and meta-heuristic algorithm to schedule DSP systems. In their work, a set of operators' arrangement forms chromosomes that are updated through some evolutionary actions such as mutation, selection, and crossover. The fitness function at each stage has the duty of evaluating the performance metrics of the new generations.

(Peng et al., 2015) have modeled the scheduling problem as a Knapsack problem. The amount of resource usage of tasks and available capacity is determined by the user. The goal of scheduling is to reduce communication overheads while the scheduler tries to provide resources for tasks in the topology. In both of the aforementioned works (Smirnov et al., 2017; Peng et al., 2015), the user should have enough knowledge about the essence of tasks. Also, when a system faces a heavy workload, it cannot handle or reschedule the tasks.

Traffic-aware scheduling has been investigated widely in (Aniello et al., 2013; Xu et al., 2014; Liu & Buyya, 2017). T-storm (Xu et al., 2014) aims to minimize communication at two inter-node and inter-process communication levels. At runtime, T-storm adjusts the number of worker nodes and the number of slots (JVMs). However, due to the lack of ability to determine the exact number of operators in the worker nodes, increases in the number of operators in a worker node causes overutilization, which can adversely affect the overall system response time.

(Liu & Buyya, 2017) have used the bin-packing algorithm to put as many tasks as possible in a worker node to minimize the process latencies resulting from inter-node communications. The MAPE loop monitors the performance metric and, when it finds a better scheduling plan, reschedules the nodes of the cluster. A major difficulty with this solution is determining the exact threshold for the number of tasks assigned to a worker node. Overutilization of a worker requires rescheduling of tasks that could well reduce the overall system performance due to rescheduling downtime and state migration.

(Aniello et al., 2013) have devised a two-level scheduler; before running the topology, it considers nothing about operators' communication patterns and the load of worker nodes. At runtime, a second

scheduler permanently monitors the transferred tuples between executors and decides whether changes in the assignments are necessary. The difficulty with this scheduler is its initial scheduling, which has no estimation about worker nodes' computational capacity, which will cause many changes in operator assignments at runtime.

(Heinze et al., 2014) have developed an elastic scaling platform that tries to maximize the system's utilization and keep the system response time under a certain threshold using an operator migration technique. Firstly, this strategy requires movements of operators in case of heavy workloads, which causes overutilization and increases the topology response time. Besides, it cannot be used in heterogeneous clusters due to the differences between worker nodes.

(Eskandari et al., 2020) have proposed I-Scheduler, which tries to minimize inter-node communications to increase overall throughput. I-Scheduler leverages K-way graph partitioning to find the most communicating tasks. The graph size is reduced by combining these tasks and repeating it until all partitions reach a predefined size. The smaller size of partitions enables the optimization algorithms to assign tasks to the worker nodes based on their capacity. However, I-Scheduler is heavily reliant upon the CPLEX as an optimization software. Furthermore, the defined threshold for the size of partitions is empirical and for the heterogeneous worker nodes only targets the user-defined capacity of nodes rather than their computational status.

3. Design and implementation

To present our proposed SP-Ant scheduler proper in this section, we first give a brief introduction to the Ant Colony Optimization algorithm, followed by a problem statement and our formulation of the stated problem.

3.1. Ant colony optimization

Ant Colony Optimization (ACO) was introduced by Marco Dorigo in 1992. This algorithm is a population-based and multi-agent technique inspired by the behavior of ants in the real world. It was originally applied to the Traveling Salesman Problem. Some complex optimization problems such as graphs, vehicle routing, and Internet routing have been solved using this algorithm (Dorigo et al., 2016). In addition, since scheduling is an optimization problem, ACO has been vastly used to solve the problem of scheduling and load balancing (Ragmani et al., 2019; Lin et al., 2019). ACO tends to find optimized solutions to exploit resources efficiently. ACO is a metaheuristic optimization algorithm that builds a model based on the actions of ants.

The ants of a colony leave their nest in search of food and wander across randomly. Each ant that finds food lays a pheromone trail, and this trail is gradually reinforced by other ants using this path to find food. The difference in pheromone levels between various paths helps the ants to choose a shorter path. If other ants do not use a path, the pheromone will be gradually evaporated after a while. An effective metaheuristic algorithm should concurrently use the exploration and exploitation phases. Exploration is achieved by pheromone trail management. In terms of exploitation, the pheromone level is tightly related to the quality of the solution obtained by the ants (Dorigo et al., 2006).

3.2. Problem statement

The problem of concern in this paper is scheduling in DSP systems in which executors of operators are assigned to clustered computing worker nodes. The purpose of optimization algorithms is to find an optimal solution, given the limitations and requirements of the problem. In determining the solution to a problem, there may be different solutions. To compare the solutions to a problem and select the optimal solution, a function called the objective function, also called the cost function, is defined. The choice of this function depends on the nature of the problem. For example, minimizing response time, minimizing

energy consumption and maximizing throughput are common goals in optimizing DSP systems. We aim to minimize the response time of a topology. Topology response time as a QoS metric is defined as the average time taken for the processing of a tuple by all the topology operators. Each optimization problem has a number of independent variables, which are called design variables. In our case, design variables are the network latency and the processing time of each tuple. Our objective is to minimize the response time of the topology without violating the resource constraints, namely, the available CPU and memory in each worker node. We consider our cluster a heterogeneous one consisting of a set of worker nodes with different computational power. Firstly, operators are as divergent as they have different computational complexities. Secondly, in a heterogeneous cluster, due to the computational power differences between each node, running tasks on each node would result in different throughput. The scheduler should decide whether it is beneficial to collect executors in a worker node to reduce communication latency or scatter the executors to different nodes to exploit more computational power. The scheduling is in the form of an optimization problem. We aim to use ACO as an evolutionary algorithm to address this problem through several iterations.

3.3. Problem formulation

We model the SP-Ant scheduling by denoting a set of heterogeneous worker nodes in a cluster by $\mathcal{W} = \{w_1, w_2, \dots, w_i\}$, and $|\mathcal{W}| = i$, a set of operators in a specific topology by $\mathcal{O} = \{o_1, o_2, \dots, o_j\}$, a set of slots (worker processes) by $\mathcal{S} = \{s_1, s_2, \dots, s_k\}$ to represent a collection of slots in different the nodes, and a set of executors for a specific operator by $\mathcal{E} = \{e_1, e_2, \dots, e_t\}$, and $|\mathcal{E}| = t$. Now, the gist of the problem to solve would be to find the best $\langle e_i, s_i \rangle$ pair for all executors of topology to assign to a set of slots $\mathcal{S}$ on different nodes in a way to minimize the overall system response time. The relation between operators and executors is shown in Fig. 1. The color of executors depicts the differences in operators' time complexity, the blue executors are computation intensive, while pink executors are I/O intensive. Spreading these executors across the worker nodes in the cluster and also the number of computation or I/O intensive executors in each worker node can directly change the topology response time. For example, when locating two computation intensive executors in a specific worker node, this node may become overutilized.

Users can change the number of slots at each worker node to achieve higher performance. To assign executors, SP-Ant considers these constraints. Eq. (1) and Eq. (2) indicate that SP-Ant firstly checks the available resources of worker nodes before assigning executors. The total resources required for running executors should not exceed the capacity of worker nodes. Eq. (3) ensures that an executor is assigned to one and only one slot.

$$\sum_{e \in \mathcal{E}} \mathcal{C}_e x_{e,w} \leq \mathcal{C}_w \forall w \in \mathcal{W} \quad (1)$$

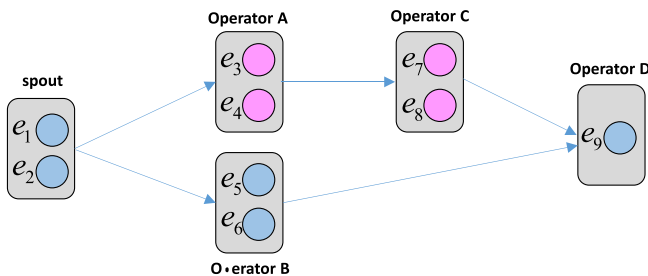


Fig. 1. Executors in a typical topology.

$$\sum_{e \in \mathcal{E}} \mathcal{M}_e x_{e,w} \leq \mathcal{M}_w \forall w \in \mathcal{W} \quad (2)$$

$$\sum_{w \in \mathcal{W}} x_{e,w} = 1 \forall e \in \mathcal{E} \quad (3)$$

$$x_{e,w} \in \{0, 1\} \forall e \in \mathcal{E}, w \in \mathcal{W} \quad (4)$$

In DSP systems, the topology response time has different definitions, but it can be considered the maximum end-to-end delay from source to sink in different paths of a DAG. The path $\mathcal{P} = \{p_1, p_2, \dots, p_n\}$ is a sequence of executors in the topology connected in the network to each other in the topology from source to sink (Cardellini et al., 2017). (Hiesl et al., 2019) have mathematically defined each path's response time by Eq. (5) wherein $\mathcal{P}_{e_i}$ accounts for processing time of each tuple by a single executor e_i and $\mathcal{L}(e_i, e_j)$ is the network latency between two connected executors.

$$\mathcal{R} = \sum_{p=1}^n \mathcal{P}_{e_p} + \sum_{p=1}^{n-1} \mathcal{L}(e_p, e_{p+1}) \quad (5)$$

Selecting the appropriate objective function is one of the most important steps in optimization algorithms. The ACO algorithm tries to minimize response time $\mathcal{R}$ as an objective function. From one side, each executor's processing time is tightly related to the time complexity of executors and the computational power of the worker node that hosts the executor. On the other side, the scheduler should select whether it is beneficial to assign a number of executors to one worker node or give a penalty for increasing the network latency and spread executors to different worker nodes.

For the sake of simplicity, we assign executors of Spouts to pre-defined slots because these types of components must be adjusted by the user for input of data and do not participate in our scheduling. All

Table 1
Notations Used in the Scheduling Algorithms.

Symbol	Meaning
$\mathcal{W}$	A set of worker nodes
$\mathcal{O}$	A set of operators in a topology
$\mathcal{E}$	A set of executors of operators in topology
$\mathcal{S}$	A set of slots on worker nodes
$\mathcal{E}_{ACO}$	A set of executors participating in ACO
$x_{e,w}$	A control Variable
$\mathcal{C}_{w_i}$	Available CPU capacity of a worker node
$\mathcal{M}_{w_i}$	Available Memory capacity of a worker node
$\mathcal{C}_{e_i}$	CPU usage of an executor
$\mathcal{M}_{e_i}$	Memory usage of an executor
$\mathcal{C}_{o_i}$	CPU usage of an operator
$\mathcal{R}$	Topology response time
$\mathcal{P}_e$	Executor processing time
$\mathcal{L}(e_i, e_{i+1})$	Network delay of two connected executors
$\mathcal{N}_{ant}$	Number of ants in ACO
$\mathcal{Ant}_k$	An ant from a set of defined ants
$\mathcal{A}_k$	A set of assignments by each ant
e_i^j	Executor i assigned to slot j
s_p	A randomly selected slot
$\mathcal{F}$	Cost function
$\mathcal{M}_{max}$	Maximum number of iterations in ACO
τ	Pheromone information
η	Heuristic information
α	Importance of pheromone information
β	Importance of heuristic information
ρ	Constant for evaporation rate
$\mathcal{T}_c$	Threshold of assigned operators to nodes

notations we use hereafter in the paper are summarized in Table 1. The entire process of scheduling by the SP-Ant is shown in Fig. 2.

3.4. The SP-Ant scheduling algorithm

The ACO algorithm uses all feedbacks of heuristic information to reach a global optimization. The optimal solution is gradually obtained by leveraging probabilities that are exploited from heuristic and pheromone information (Ragmani et al., 2019; Lin et al., 2019; Arunarani et al., 2019).

Heuristic information and pheromone level are two building blocks of the ACO algorithm that should be updated through various iterations to pave the way for making future decisions.

3.4.1. Initial scheduling

To increase the efficiency of the ACO algorithm in terms of topology response time and time complexity of the algorithm, we have changed the number of input operators to the algorithm. SP-Ant combines a bin-packing algorithm as its initial scheduling with the ACO algorithm. The bin-packing algorithm's goal is to reduce the communication latency between executors and, furthermore, to curb the number of executors participating in the evolutionary algorithm, to reduce the ACO convergence time.

Firstly, by considering the amount of communication, SP-Ant sorts operators in descending order and sorts worker nodes in a descending manner based on their available computational resources in a point-based system. (Peng et al., 2015) have used a point-based system where the amount of available CPU and memory of each worker node is designated by points.

As shown in Fig. 3, we have introduced a user-defined threshold $\mathcal{T}_c$ as a percentage of the CPU capacity of the worker nodes. All operators in w_i are sorted in descending order based on their communications. In this algorithm, a number of operators on each worker node with the highest communication and with the summation of CPU usage equal to this percentage $\mathcal{T}_c$ will be fixed in the worker nodes, and other operators

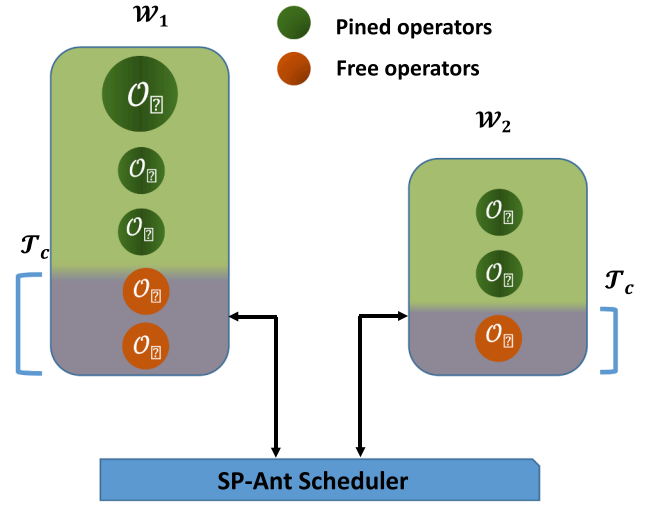


Fig. 3. The scope of the SP-Ant scheduler.

with low communication will get scheduled by the ACO algorithm.

Algorithm 1: Determining operators participating in the ACO Algorithm

```

1: for each  $w_i \in \mathcal{W}$  in  $N$  do
2:   for each  $o_i \in \mathcal{O}$  in  $w_i$  do
3:     if  $j + \mathcal{C}_{o_i} < \mathcal{T}_c \times \mathcal{C}_{w_i}$ 
4:        $j \leftarrow j + \mathcal{C}_{o_i}$ 
5:       Pin  $o_i$  in  $w_i$ 
6:     else
7:       Add  $o_i$  to set  $\mathcal{E}_{ACO}$ 
8:     end if
9:   end for
10: end for
11: All operators  $o_i$  in  $\mathcal{E}_{ACO}$  will be scheduled by ACO.

```

For example, when $\mathcal{T}_c = 60$ percent, SP-Ant pins the operators with the highest communication and aggregation of CPU utilization equal to 60

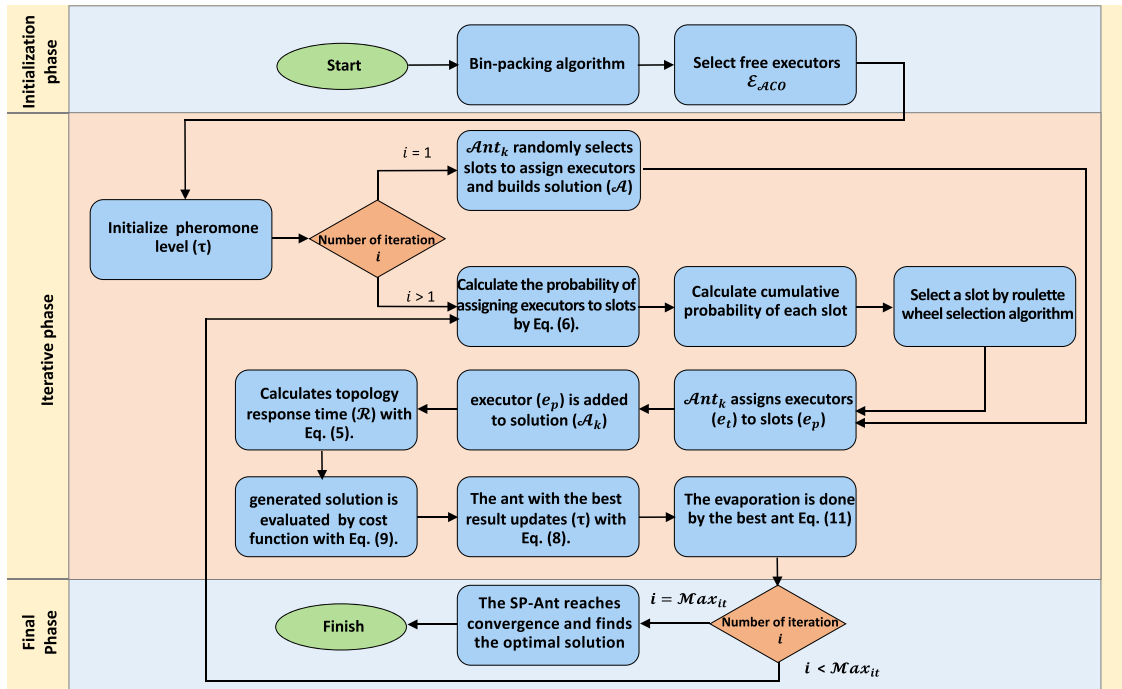


Fig. 2. The flowchart of SP-Ant algorithm.

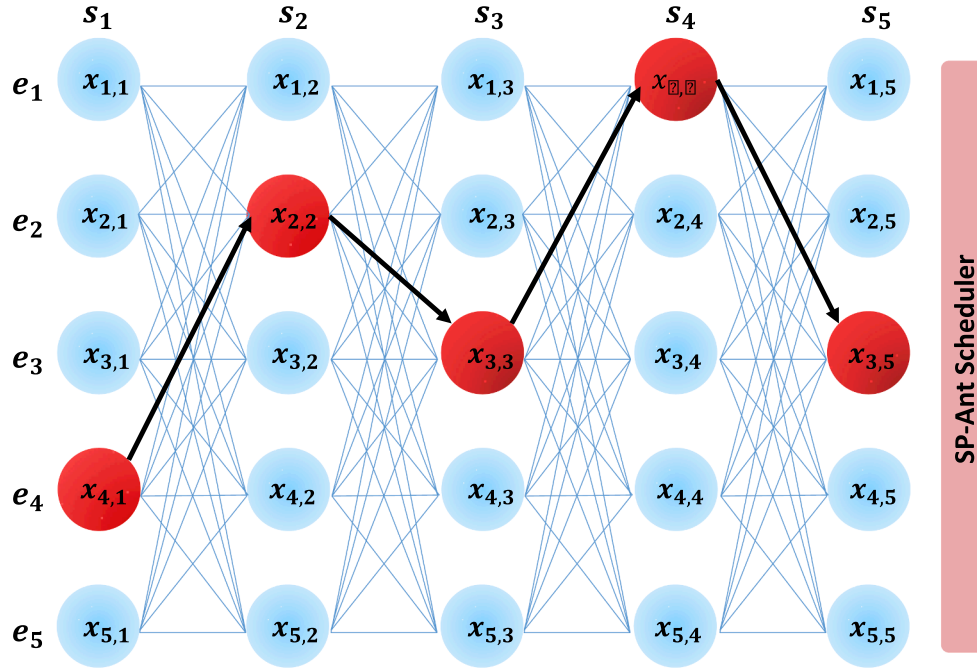


Fig. 4. Assignment of executors to slots by SP-Ant.

percent in the worker node, and the remaining operators are scheduled by the SP-Ant scheduler. The SP-Ant scheduler forces the executors of operators with high communication latencies to stay in a bin (worker nodes) near each other in order to decrease the topology response time.

3.4.2. Executor assignment by ACO

The SP-Ant scheduler uses the ACO algorithm to assign executors to worker nodes by following the steps below:

(1) As $|\mathcal{S}| = j$, the pheromone level matrix is initially set to $1/j$. Assigning equal values in the pheromone matrix guarantees a proper random movement of ants in the first stage. In the first iteration, ants move randomly with equal probability and only update the pheromone matrix.

(2) In SP-Ant, $\mathcal{Ant}_k$ is an agent trying to build a new solution based

on the previously acquired knowledge. In the first iteration, ants randomly select slots and assign executors $e_i \in \mathcal{E}$ to $s_j \in \mathcal{S}$. An $\mathcal{Ant}_k$ builds a solution, which is a set with a sequence of assignments $\mathcal{A} = \{a_1, a_2, \dots, a_t\}$. Each $a_i \in \mathcal{A}_k$ is an assignment $a_i e_i \rightarrow s_j$ by $\mathcal{Ant}_k$.

Regarding the assignment of executors, the scheduler can adopt two different approaches. Firstly, after assigning each executor to a certain slot, SP-Ant can put it in an unallowed list, prohibiting other ants from assigning executors to this slot. Secondly, the scheduler can let various ants to assign executors to a specific slot in each iteration. In this paper, the SP-Ant scheduler adopts the second approach to reduce communication latencies between executors. We postpone the investigation of the former approach for future works.

For the sake of clarity, we explain our chosen approach via an example. Let the set of assignments by $\mathcal{Ant}_k$ be $\mathcal{A}_k = \{e_1^5, e_2^3, e_3^3, e_4^2, e_5^1\}$

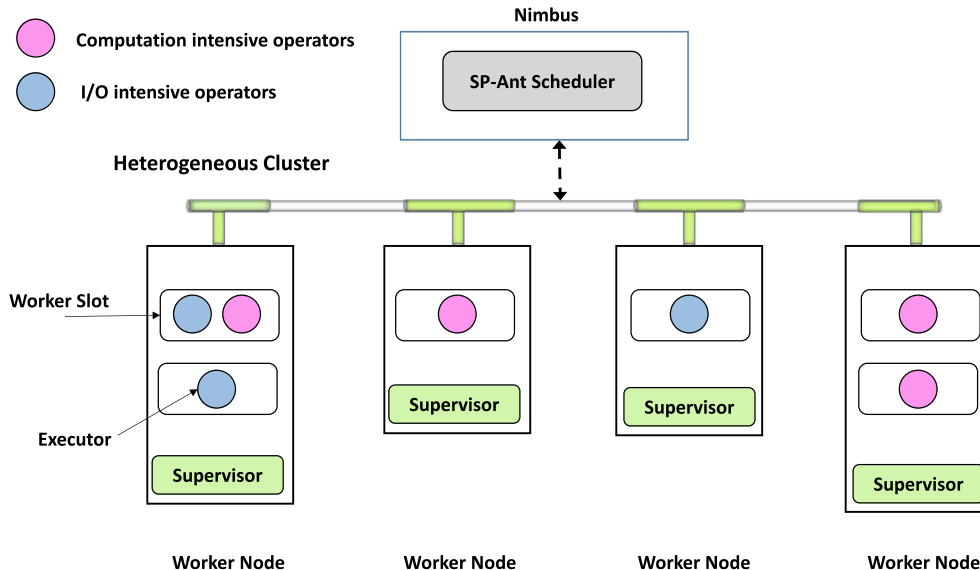


Fig. 5. Assignment of executors across worker nodes by SP-Ant.

wherein, e_i^k symbolizes assigning executor i to slot k in a way that in this set, some slots are repetitive, or a number of them are defined as slots for one worker node. (Liu et al., 2014) have used a scheme using a matrix to display the map of movements of ants and illustrated the structure of the solutions. Accordingly, based upon this structure, we have modeled executors' assignments by ants in the SP-Ant scheduler, as shown in Fig. 4. In this matrix, the map of executors' assignments to the slots by each ant is shown by red dots. The cost function evaluates this path, and if this assignment plan improves the topology response time, it would be followed and reinforced by other ants.

(3) In other iterations, Ant_k for selecting a slot, should calculate the probability of assigning executors to worker nodes. Firstly, Ant_k needs to acquire the probabilities based on the previous information using Eq. (6).

$$p_k(e_i, s_j) = \frac{\tau(e_i, s_j)^\alpha \eta(e_i, s_j)^\beta}{\sum_{u \in S} \tau(e_i, u)^\alpha \eta(e_i, u)^\beta} \quad (6)$$

In Eq. (6), τ represents the pheromone level matrix, and η symbolizes the heuristic information. Also $\alpha \geq 0$ indicates the degree of importance of pheromone, and $\beta \geq 1$ shows the importance of heuristic information. In addition, $\mathcal{U}$ refers to all possible choices that can be selected by Ant_k for hosting executor e_i .

ACO uses the roulette wheel selection algorithm to calculate the probabilistic choice of next steps by Ant_k . Each probability value is a slice on the roulette wheel, and the algorithm selects an item proportional to its probability or size of the slice (Dorigo et al., 2006; Goldberg & Deb, 1991). Although greater possibilities have a higher chance of being selected, the weaker ones can survive too (Algorithm 2 line 10–29). This characteristic provides both exploitation from previous information and exploration of new worker nodes for the SP-Ant scheduler.

Ant_k (4) repeats the process of selecting worker nodes to assign executor $|E| = t$ times, and as a result, the set of $\mathcal{A}_k$ is generated, which is a solution or path in the topology. The cost function $\mathcal{F}$ evaluates this assignment by measuring topology response time $\mathcal{R}$ by Eq. (5). Moreover, the heuristic value is obtained by Eq. (7).

$$\eta_{e_i, s_j} = \frac{1}{\mathcal{R}} \quad (7)$$

Since for assigning of executors we use the ant colony system (ACS) algorithm, after k ants finish their iteration, only the ant with the best result in an iteration can update the pheromone level according to Eq. (8).

$$\tau_{e_i, s_j} = \tau_{e_i, s_j} + \Delta \tau_{e_i, s_j}^{(k)} \quad (8)$$

$$\Delta \tau_{e_i, s_j}^{(k)} = \frac{Q}{L_k} \quad (9)$$

$$L_k = \Delta \mathcal{R} \quad (10)$$

In Eq. (9), Q is defined to find high-quality solutions with low cost, and here we set it to 2, experimentally, and L_k is equal to the changes in topology response time as stated in Eq. (10).

(5) Once the ants in each iteration complete their assignments, the best ant can update the pheromone level and calculate the pheromone's evaporation using Eq. (11). In this equation, ρ is a constant number. The evaporation in ACO sets the stage for correction of wrong scheduling decisions, and the assignments would be corrected in a number of iterations.

$$\tau_{e_i, s_j} = (1 - \rho) \tau_{e_i, s_j} \forall (i, j) \in E, S \quad (11)$$

Having gone through all the aforementioned 5 steps, the SP-Ant algorithm finally determines various executors' assignments across the worker nodes, as shown in Fig. 5. As operators' time complexity is diverse and some of them may be computationally intensive or I/O

intensive, SP-Ant tries to exploit the computation power of various worker nodes by assigning operators to different worker nodes.

Algorithm 2: Using the ACO Algorithm in the SP-Ant Scheduler

Input: A set of executors of topology derived by Algorithm1

Output: An assignment of executors to slots that minimizes the response time of topology

```

1: initial matrix  $\tau \leftarrow 1 / |\mathcal{S}|$ 
2: for  $i \leftarrow 1$  to  $Max_{it}$  do
3:   for  $k \leftarrow 1$  to  $N_{ant}$  do
4:     for  $t \leftarrow 1$  to  $|\mathcal{E}_{ACO}|$  do
5:       if  $(i = 1)$ 
6:          $\mathcal{P} \leftarrow$  Generate a random number  $\in [1, |\mathcal{S}|]$ 
7:          $Ant_k$  assigns  $e_t$  to  $s_p$ 
8:         Add  $e_t^p$  to set  $\mathcal{A}_k$ 
9:       else
10:      for  $j \leftarrow 1$  to  $|\mathcal{S}|$  do
11:        Probability  $\leftarrow$  Calculate  $p_k(e_t, s_j)$  with Eq. (6)
12:        if  $(j = 1)$ 
13:          cumulative  $[j] \leftarrow$  probability
14:        else
15:          for  $(r \leftarrow 1$  To  $r \leq j)$ 
16:            sum  $\leftarrow$  sum + probability
17:          end for
18:        total  $\leftarrow$  sum + probability
19:        cumulative  $[m] \leftarrow$  total
20:      end if
21:       $x \leftarrow$  Generate a random number  $\times E[0,1]$ 
22:      for  $(m \leftarrow 0, m < \text{size}(\text{cumulative}))$ 
23:        if  $(x < \text{cumulative}[m])$ 
24:           $p \leftarrow m$ 
25:          break
26:        end if
27:      end for
28:      if  $p$  is found break
29:    end for
30:  end if
31:   $Ant_k$  assigns  $e_t$  to  $s_p$ 
32:  Add  $e_t^p$  to set  $\mathcal{A}_k$ 
33: end for
34: Calculate  $\mathcal{R}$  using Eq. (5)
35: Evaluate  $\mathcal{A}_k$  with cost function
36: Calculate heuristic  $\eta$  using Eq. (7)
37: end for
38: Best ant's result updates  $\tau$  using Eq. (8)
39: Best ant's result updates evaporation using Eq. (11)
40: end for
41: Select the best scheduling assignment with minimum latency

```

The SP-Ant time complexity relates to the number of executors' assignments and updating relevant pheromone matrix $\mathcal{O}(2 \times |\mathcal{E}|)$. It is clear that since a subset of executors contributes to the scheduling, the final time complexity would be $\mathcal{O}(\mathcal{T}_c \times 2 \times Max_{it} \times N_{ant} \times |\mathcal{E}|)$, wherein Max_{it} and N_{ant} denote the maximum number of iterations in the ACO algorithm and the number of ants, respectively.

4. Implementation and experimental results

SP-Ant was implemented by extending the standard Apache Storm release (2.1.0). Different modules such as Storm-Server has changed in order to implement the SP-Ant scheduler. In addition, we made the source code of SP-Ant system publicly available at: <https://github.com/SP-Ant/HPC>. A series of test scenarios were designed to evaluate the performance of SP-Ant using a standard benchmark topology. The word count topology is a MapReduce topology, including 4 elements: 1 spout and 4 bolts (Xu et al., 2014). A sentence generator is a spout operator that continuously reads strings from a text file¹ with 260,000 sentences and pushes them into the topology. At the end of the file, it repeats from the beginning. Splitter, counter, and log writer are bolts

¹ <https://www.gutenberg.org/ebooks/11>.

that form the topology. Splitter separates sentences into words, and the counter increases the counter variable by distinguishing each particular word. Finally, the log writer writes the output into log files. The sentence generator controls the input data rate to the topology by calling the sleep function for specific milliseconds. The average input workload was 3000 tuples/second. This topology has 2 instances of sentence generator, and each operator has 5 executors. The structure of word count topology is shown in Fig. 6.

In terms of test environment, a number of virtual machines (VMs) with heterogeneous computational power and capacity were used in the underlying cluster, notwithstanding that the SP-Ant scheduler can cover both heterogeneities in the CPU and memory architecture, thanks to the evolutionary scheduling algorithm acquiring near-real estimate about the status of worker nodes in the subsequent iterations.

The underlying cluster has 5 VMs with various numbers of cores of Dell PowerEdge R710 Server 2x Intel Xeon X5650 with 12 cores and 32 GB of DDR3 RAM. Virtual machines are divided into 3 computational capacity categories, namely, high (1 VMs), medium (2 VMs), and low (2 VMs). In the high-capacity configuration, each VM has 4 cores and 4 GB of memory, whereas that of the medium-capacity VMs have 2 cores and 3 GB of memory and VMs with low-capacity configuration have 1 core and 2 GB of memory. Also, a physical server with 12 cores and 16 GB of memory is dedicated to the Nimbus and Zookeeper. Ubuntu 18.04.LTS Server has been used as an operating system. We have implemented SP-Ant by extending standard Apache Storm (2.1.0).

We have compared the experimental results of SP-Ant algorithm with those for two different built-in scheduling algorithms of Apache Storm, namely, default scheduler and R-Storm. The Storm's default scheduler uses a round-robin algorithm that spreads all tasks across the cluster without any consideration about the task requirements and predefined priority in the processing. R-Storm is a resource-aware scheduler wherein users should specify the processing requirements of each task. R-Storm then decides which node has such a capacity. Both of these schedulers heavily rely on the knowledge of users.

Fig. 7 shows that at the start of the experiment, the topology response time of the SP-Ant algorithm was about 25 ms, but after several iterations, it reached convergence, and during the remaining time of the experiment, SP-Ant showed a stable topology response time around 7 ms and outperformed the other two algorithms. In this experiment, we set $\alpha = 1$, $\beta = 1$, and $\rho = 0.5$.

To explain this behavior, it is clear that in the long run, the scheduler can use its knowledge, and as a result, distinguish the best assignment of executors to worker nodes. R-Storm and the default scheduler of Storm blindly scheduled tasks. R-Storm only considered the communication pattern and the resources needed for the topology and, therefore, by a simple variation in workload, performance could be degraded in case worker nodes become over utilized.

Fig. 8 illustrates the result of the investigation of the average execution latency of a selected Bolt in a topology in different scheduling algorithms. The average execute latency relates to the average time needed to process a tuple by a particular Bolt. SP-Ant had a lower rate of delay in the process of each tuple in comparison to R-Storm and the default scheduler of Storm. At the start of the experiment, the rate was higher than R-Storm, but after some evolutionary actions, the rate of SP-Ant significantly decreased.

According to Fig. 9, the average topology response time for the SP-Ant algorithm outperformed a different version of the SP-Ant

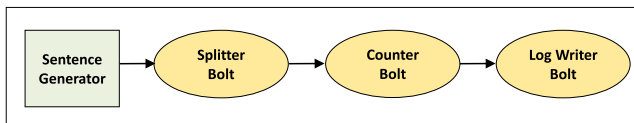


Fig. 6. Word count topology.

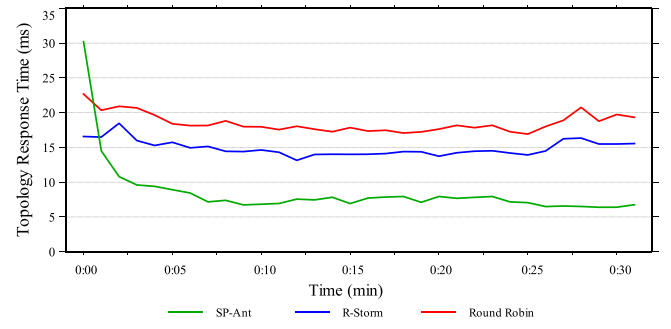


Fig. 7. Performance comparison of SP-Ant with other scheduling algorithms.

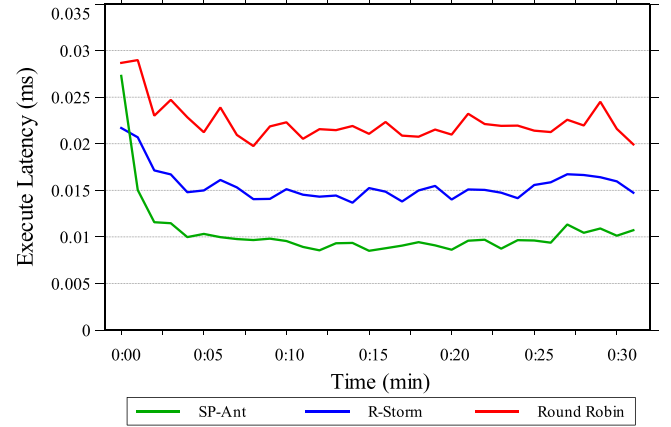


Fig. 8. Bolt execute latency in different scheduling algorithms.

algorithm without using initial bin-packing by at least 13 percent when $\mathcal{T}_c = 40$. By contrast, although higher values of $\mathcal{T}_c$ decreased the communication overheads, but degraded the system performance since some worker nodes might have become overutilized and no longer used computational resources of the cluster, decreasing the chance of finding the best worker nodes by the ACO algorithm.

We performed further experiments by changing the value of parameters α and β . As shown in Fig. 10, the overall topology response times were close to each other. However, increasing the value of β or heuristic information caused a slight reduction in topology response time. By contrast, the higher value of α or importance of pheromone level had an adverse effect on topology response time. It is worth noting that the system administrator, by considering the number of ants and iterations, would be able to change configurations of these parameters to achieve the highest level of efficiency.

As the SP-Ant algorithm uses a meta-heuristic algorithm, each run may have different solutions and consequently different response times. Fig. 11 shows the results in the various experiments as we have run the SP-Ant algorithm 10 times. Based on the measurements, after the convergence in the scheduling algorithm, topology response times were very close to each other. These differences are associated with the first selection of nodes, in some cases as a result of random selection, the topology response time would be much lower. ACO for the first steps of the algorithm uses a random function, and it does not have any knowledge about the essence of operators and the status of worker nodes.

The SP-Ant algorithm reaches convergence through a number of iterations by getting feedbacks from previous assignments. For running the ACO algorithm, a number of ants should be defined to perform assignments. In this experiment, we set the number of ants to 6. We monitored the number of assignments in order to reach the final scheduling map. Fig. 12 shows that similar to topology response time,

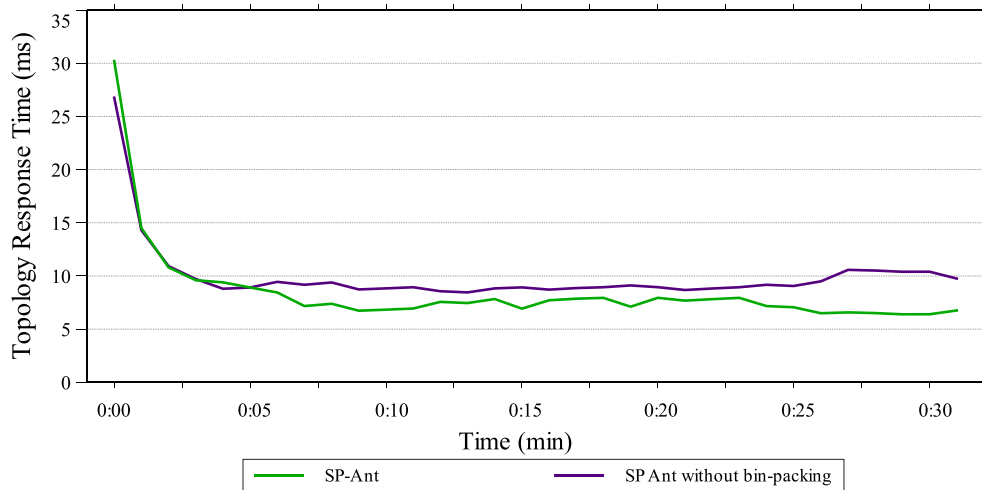


Fig. 9. Performance comparison of initial bin-packing in SP-Ant.

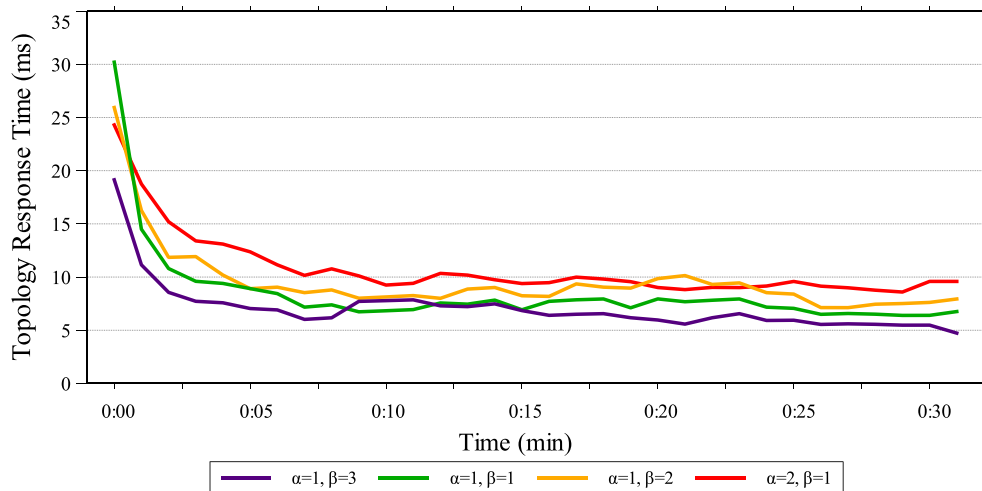


Fig. 10. Performance comparison of different configurations of parameters in the SP-Ant algorithm.

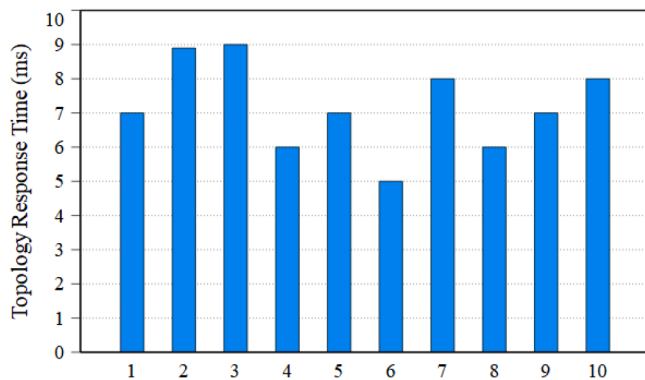


Fig. 11. Average topology response time in different runs of the SP-Ant algorithm.

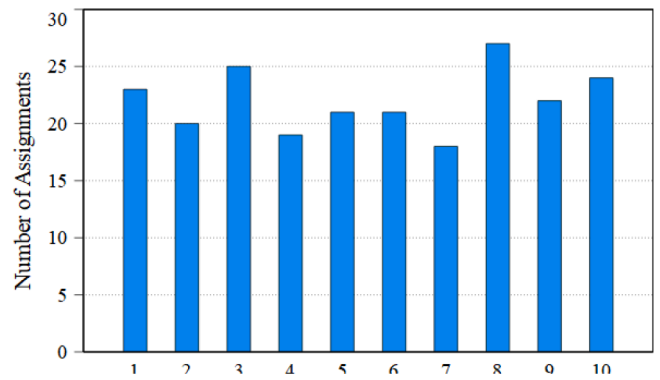


Fig. 12. Required number of assignments to complete the scheduling in different runs of the SP-Ant algorithm.

the number of assignments was near each other. After nearly an equal number of assignments, the SP-Ant algorithm completed the task assignment successfully.

5. Discussion

Before running SP-Ant, the scheduler considers no assumptions about operators' time complexity and the computational power of worker nodes. In terms of local view, it is clear that in a heterogeneous

cluster, each operator's execute latency is related to the complexity of the operator's code and the status of the host node, which is only discernable by the scheduler while running operators on worker nodes. Concerning the global view, a set of various assignments in nodes and communication between operators affects the overall topology response time, which in turn sets the stage to select an optimum plan of deployment of operators through various iterations. Users no longer need to perceive the operator's requirement as the SP-Ant scheduler provides a real estimation by assigning operators. SP-Ant can work without any initial scheduling. The bin-packing algorithm only accelerates the convergence time and considers the inter-node communication, which in turn causes a reduction in topology response time.

Our experiments showed that SP-Ant takes some time to assign operators to nodes to complete its iterations, find the most efficient assignments, and outperform other counterpart available schedulers. In addition, the result of the investigation of the execute latency of a certain operator showed that SP-Ant tries to minimize topology response time by finding the propitious hosts for operators and as a result, reduce their execute latency. The SP-Ant algorithm can be easily deployed in other DSP engines such as Apache Flink and Apache Spark. Since SP-Ant aims to solve a multi-objective scheduling problem, the scheduler can well take other objectives into consideration, such as reducing the cost of deployment of operators as well as energy consumption of the cluster.

6. Conclusion

SP-Ant aims to address the problem of scheduling in distributed stream processing (DSP) systems. In a topology, operators' resource demand cannot be defined accurately by the user before running the application. Besides, different assignments of computation and I/O intensive operators to a worker node may result in different throughputs. When it comes to a heterogeneous cluster, the problem of scheduling becomes even more sophisticated. SP-Ant leverages the ACO algorithm, an evolutionary and meta-heuristic algorithm, to find operators' best assignment by considering a local optimization and simultaneously following a global optimization strategy to discover executors' desired assignment to worker nodes. SP-Ant uses the bin-packing algorithm for initial scheduling and limits the number of executors participating in the next scheduling step in order to reduce the communication overheads and time complexity of the algorithm. SP-Ant has been implemented on the standard Apache Storm. Running standard benchmarks, SP-Ant reduced response time by at least 50% on average compared to the Apache Storm's default scheduler (that uses a round-robin algorithm) and R-Storm, a resource-aware scheduler.

As future works, we intend to change the SP-Ant algorithm to address the problem of elasticity in DSP systems. The SP-Ant scheduler stops evolutionary steps after reaching convergence, however, the scheduler can postpone some evolutionary actions, and when it faces a heavy workload, it can perform new assignments to cope with the input spike of data. This strategy enables the scheduler to discover a more accurate plan of assignments by considering changes in the workload and lower the initial time of convergence. Another future improvement is to change the current cost function in a way that it can prioritize assigning the same executors of each operator to a host node, encouraging the scheduler to assign executors of operators near each other to reduce inter-node communication at runtime.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

We thank the anonymous reviewers of the journal whose comments helped us to make the revised version of paper stronger.

References

- Aniello, L., Baldoni, R., & Querzoni, L. (2013). Adaptive online scheduling in storm. In *Proceedings of the 7th ACM International Conference on Distributed Event-based Systems* (pp. 207–218).
- Arunarani, A. R., Manjula, D., & Sugumaran, V. (2019). Task scheduling techniques in cloud computing: A literature survey. *Future Generation Computer Systems*, 91, 407–415.
- Bilal, M., & Canini, M. (2017). Towards automatic parameter tuning of stream processing systems. In *Proceedings of the 2017 Symposium on Cloud Computing* (pp. 189–200).
- Cardellini, V., Lo Presti, F., Nardelli, M., & Russo Russo, G. (2018). Decentralized self-adaptation for elastic data stream processing. *Future Generation Computer Systems*, 87, 171–185.
- Cardellini, V., Grassi, V., Lo Presti, F., & Nardelli, M. (2017). Optimal operator replication and placement for distributed stream processing systems. *ACM Sigmetrics Performance Evaluation Review*, 44(4), 11–22.
- De Matteis, T., & Mencagli, G. (2017). Proactive elasticity and energy awareness in data stream processing. *Journal of Systems and Software*, 127, 302–319.
- Dorigo, M., Birattari, M., & Stutzle, T. (2006). Ant colony optimization. *IEEE Computational Intelligence Magazine*, 1, no. 4, 28–39.
- Eskandari, L., Mair, J., Huang, Z., & Eysers, D. (2020). I-Scheduler: Iterative scheduling for distributed stream processing systems. *Future Generation Computer Systems*.
- Fardbastani, M. A., & Sharifi, M. (2019). Scalable complex event processing using adaptive load balancing. *Journal of Systems and Software*, 149, 305–317.
- Goldberg, D. E., & Deb, K. (1991). A comparative analysis of selection schemes used in genetic algorithms. In *Foundations of Genetic Algorithms*, vol. 1 (pp. 69–93), Elsevier.
- Heinze, T., Jerzak, Z., Hackenbroich, G., & Fetzer, C. (2014). Latency-aware elastic scaling for distributed data stream processing systems. In *Proceedings of the 8th ACM International Conference on Distributed Event-Based Systems* (pp. 13–22).
- Hiessl, T., Karagiannis, V., Hochreiner, C., Schulte, S., & Nardelli, M. (2019). Optimal placement of stream processing operators in the fog. In *2019 IEEE 3rd International Conference on Fog and Edge Computing (ICFEC)* (pp. 1–10).
- Lin, M., Xi, J., Bai, W., & Wu, J. (2019). Ant colony algorithm for multi-objective optimization of container-based microservice scheduling in cloud. *IEEE Access*, 7, 83088–83100.
- Liu, X., & Buyya, R. (2017). D-storm: Dynamic resource-efficient scheduling of stream processing applications. In *2017 IEEE 23rd International Conference on Parallel and Distributed Systems (ICPADS)* (pp. 485–492).
- Liu, X. F., Zhan, Z. H., Du, K. J., & Chen, W. N. (2014). Energy aware virtual machine placement scheduling in cloud computing based on ant colony optimization approach. In *Proceedings of the 2014 Annual Conference on Genetic and Evolutionary Computation* (pp. 41–48).
- Lohrmann, B., Janacik, P., & Kao, O. (2015). Elastic stream processing with latency guarantees. In *2015 IEEE 35th International Conference on Distributed Computing Systems* (pp. 399–410).
- Nardelli, M., Cardellini, V., Grassi, V., & Presti, F. L. (2019). Efficient operator placement for distributed data stream processing applications. *IEEE Transactions on Parallel and Distributed Systems*, 30(8), 1753–1767.
- Osaba, E., Yang, X.-S., Fister, I., Del Ser, J., Lopez-Garcia, P., & Vazquez-Pardavila, A. J. (2019). A discrete and improved bat algorithm for solving a medical goods distribution problem with pharmacological waste collection. *Swarm and Evolutionary Computation*, 44, 273–286.
- Peng, B., Hosseini, M., Hong, Z., Farivar, R., & Campbell, R. (2015). R-storm: Resource-aware scheduling in Storm. In *Proceedings of the 16th Annual Middleware Conference* (pp. 149–161).
- Precup, R.-E., David, R.-C., Petriu, E. M., Szedlak-Stinean, A.-I., & Bojan-Drăgos, C.-A. (2016). Grey wolf optimizer-based approach to the tuning of PI-fuzzy controllers with a reduced process parametric sensitivity. *IFAC-PapersOnLine*, 49(5), 55–60.
- Ragmani, A., Elomri, A., Abghour, N., Moussaid, K., & Rida, M. (2019). FACO: A hybrid fuzzy ant colony optimization algorithm for virtual machine scheduling in high-performance cloud computing. *Journal of Ambient Intelligence and Humanized Computing*, 1–13.
- Röger, H., & Ruben, M. (2019). A comprehensive survey on parallelization and elasticity in stream processing. *ACM Computing Surveys (CSUR)*, 52(2), 1–37.
- Russo, G. R., Cardellini, V., & Presti, F. L. (2019). Reinforcement learning based policies for elastic stream processing on heterogeneous resources. In *Proceedings of the 13th ACM International Conference on Distributed and Event-based Systems* (pp. 31–42).
- Smirnov, P. A., Melnik, M., & Nasonov, D. A. (2017). Performance-aware scheduling of streaming applications using genetic algorithm. In *ICCS* (pp. 2240–2249).

- Stanoi, I., Mihaila, G., Palpanas, T., & Lang, C. (2007). Whitewater: Distributed processing of fast streams. *IEEE Transactions on Knowledge and Data Engineering*, 19 (9), 1214–1226.
- Xu, J., Chen, Z., Tang, J., & Su, S. (2014). T-storm: Traffic-aware online scheduling in Storm. In *2014 IEEE 34th International Conference on Distributed Computing Systems* (pp. 535–544).
- Zapata, H., Perozo, N., Angulo, W., & Contreras, J. (2020). A hybrid swarm algorithm for collective construction of 3D structures. *International Journal of Artificial Intelligence*, 18(1), 1–18.



Mohammadreza Farrokh received his M.Sc. degree in Computer Engineering from Iran University of Science and Technology, in 2020, with a thesis about elastic and resource-aware scheduling in distributed data stream processing systems. Currently, he works as a research assistant in the Distributed Systems Laboratory at the School of Computer Engineering, Iran University of Science and Technology. His main interests include distributed systems, embedded systems, stream processing, big data analytics.



Hamid Hadian received his M.Sc. degree in Computer Engineering from Iran University of Science and Technology, in 2020, with a thesis about scheduling and operator placement in distributed data stream processing systems. Currently, he works as a research assistant in the Distributed Systems Laboratory at the School of Computer Engineering, Iran University of Science and Technology. His research focuses on improving the performance of stream processing systems and using incremental learning in scheduling. His interests include distributed systems, stream processing, parallel computing, and incremental learning.



Mohsen Sharifi is a Professor of System Software Engineering at the School of Computer Engineering of Iran University of Science and Technology. He received his B.Sc., M.Sc., and Ph.D. in Computer Science from the University of Manchester in the United Kingdom. He directs a Distributed Systems research group and laboratory. His main interest is the development of distributed systems, solutions, and applications, particularly for use in various fields of science. He has developed *c-sharifi* a high-performance, scalable cluster solution comprising of any number of commodity computers for use in scientific applications requiring high-performance and availability. The development of a true Distributed Operating System is on top of his wish list.



Ali Jafari received his M.Sc. degree in Computer Engineering from Iran University of Science and Technology in 2017. He is currently a Ph.D. student in Computer Engineering and a member of the Distributed Systems Laboratory at the School of Computer Engineering, Iran University of Science and Technology. His research interests include Distributed Systems, Stream Processing, and Complex Event Processing.

Update

Expert Systems With Applications

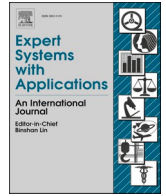
Volume 250, Issue , 15 September 2024, Page

DOI: <https://doi.org/10.1016/j.eswa.2024.123968>



Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa

Corrigendum

Corrigendum to “SP-ant: An ant colony optimization based operator scheduler for high performance distributed stream processing on heterogeneous clusters” [Expert Syst. App. 191 (2022) 116322]

Mohammadreza Farrokh, Hamid Hadian, Mohsen Sharif^{*}, Ali Jafari*Distributed Systems Research Lab., School of Computer Engineering, Iran University of Science and Technology, Tehran, Iran*

The authors regret that they did not check the text of their affiliations during submission and proof corrections. Currently the affiliation written on the first page of the article is “Distributed Systems Research, Lab University of Science and Technology, Tehran, Iran”. The correct affiliation is “Distributed Systems Research Lab., School of Computer Engineering, Iran University of Science and Technology, Tehran, Iran”. Given there are other universities with the name of “University of

Science and Technology “, our article has not been indexed in Iran University of Science and Technology. The authors highly appreciate the correction of the affiliation to “Distributed Systems Research Lab., School of Computer Engineering, Iran University of Science and Technology, Tehran, Iran”.

The authors would like to apologise for any inconvenience caused.



DOI of original article: <https://doi.org/10.1016/j.eswa.2021.116322>.

^{*} Corresponding author.

E-mail addresses: Mo_farrokh@comp.iust.ac.ir (M. Farrokh), hadian_hamid@cmps2.iust.ac.ir (H. Hadian), msharifi@iust.ac.ir (M. Sharif), ali_jafari@comp.iust.ac.ir (A. Jafari).

<https://doi.org/10.1016/j.eswa.2024.123968>

Available online 6 May 2024

0957-4174/© 2024 Elsevier Ltd. All rights reserved.